

CalveKV: Cross-Layer Adaptive Latent Vector Eviction for Efficient LLM Inference

Anonymous submission

Abstract

Despite the exceptional capabilities of Large Language Models (LLMs) in handling long contexts, the memory overhead associated with Key-Value (KV) caching during autoregressive inference remains a primary bottleneck for high-throughput deployment.

Recent architectural innovations—specifically the Multi-Head Latent Attention (MLA) mechanism introduced in DeepSeek-V2/V3—have significantly alleviated this issue by compressing KV states into low-rank latent vectors. However, generation tasks involving ultra-long contexts still demand further memory optimization. Existing token eviction strategies primarily target uncompressed, traditional KV caches; applying them directly to MLA would necessitate decompressing the latent space, thereby negating MLA’s inherent memory efficiency advantages. To address this, we propose "CalveKV," a novel, highly generalizable, and scalable algorithm that operates efficiently directly within the compressed latent space.

Our experiments reveal a phenomenon of "Cross-layer Persistence" regarding latent vector importance: the importance of latent vectors determined in shallower Transformer layers serves as a reliable prior for attention distributions in deeper layers. Specifically, by applying a composite scoring function based on the L2-norm and variance of latent features at a designated early decision layer (e.g., Layer 4), our algorithm enables the safe, global physical truncation of redundant latent vector caches across all subsequent layers. Extensive experiments on long-text and code corpora demonstrate the method’s effectiveness. Latent Eviction introduces negligible computational overhead; it further compresses peak memory usage to 50–60 percent of the original MLA baseline while maintaining Time-to-First-Token (TTFT) and prefill latency. Crucially, this extreme compression does not compromise model accuracy, maintaining robust retrieval performance in "Needle In A Haystack" (NIAH) evaluations. Code and artifacts are withheld for double-blind review and will be released upon acceptance.

Introduction

Large Language Models (LLMs) have rapidly expanded the practical context length of neural text generation, enabling

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

stronger performance on retrieval-heavy reasoning, long-form code completion, and document-grounded generation. Yet, for autoregressive decoding, the memory footprint of the Key-Value (KV) cache remains a central systems bottleneck. As sequence length grows, KV states scale linearly with the number of processed tokens and become the dominant memory consumer, directly constraining batch size, throughput, and deployability under fixed GPU budgets.

Architectures such as DeepSeek-V2/V3 partially mitigate this bottleneck through Multi-Head Latent Attention (MLA), which stores compressed latent representations instead of conventional full KV tensors. While MLA substantially improves memory efficiency, ultra-long context generation still faces strict memory limits in realistic serving scenarios. A natural direction is token eviction, but most existing eviction methods are designed for uncompressed KV caches. Applying those strategies to MLA generally requires decompression or proxy reconstruction, which undermines the primary efficiency advantage of latent-space caching.

This paper introduces **CalveKV**, a latent-native eviction framework that performs global cache reduction directly in the compressed space. The key observation is a cross-layer persistence effect: token importance estimated from latent statistics in shallower layers provides a reliable prior for deeper-layer attention allocation. Based on this observation, CalveKV computes a composite importance score from latent feature norm and variance at an early decision layer, then executes safe physical truncation of low-importance latent vectors for all subsequent layers. This design avoids repeated per-layer re-scoring and avoids decompressing latent caches.

CalveKV is designed for deployment efficiency rather than architectural redesign. It is model-compatible, lightweight, and easy to integrate into existing MLA inference pipelines. Empirically, on long-text and code-centric workloads, CalveKV reduces peak cache memory to approximately 50–60% of the MLA baseline, while preserving Time-to-First-Token (TTFT), prefill latency, and downstream retrieval quality (including Needle-In-A-Haystack settings).

Our main contributions are as follows:

- We identify and validate *cross-layer persistence* of latent token importance in MLA-based Transformers, providing a practical signal for early-layer cache decisions.
- We propose **CalveKV**, a compressed-space eviction algorithm that combines latent norm and variance into a

unified score and performs globally consistent truncation across later layers.

- We show that substantial memory compression can be achieved with negligible runtime overhead and without measurable degradation in long-context retrieval behavior.

These results suggest that latent-space eviction is a viable path toward scalable long-context LLM serving, complementing architectural compression with systems-level cache control.

Background and Related Work

The attention mechanism utilized in Transformers continues to serve as the predominant cornerstone of contemporary large language models, with the corresponding KV cache compression emerging as a frequently debated topic in current research. The MLA algorithm represents a novel and highly efficient compression approach.

Multi-Head Latent Attention

DeepSeek-V2/V3 (DeepSeek-AI 2024, 2025) introduce *Multi-Head Latent Attention* (MLA) to alleviate the KV-cache bottleneck through a low-rank bottleneck projection. Rather than caching full-rank key and value tensors, MLA compresses each hidden state into a compact *KV latent vector*:

$$\mathbf{c}_t^{KV} = W^{DKV} \mathbf{h}_t, \quad \mathbf{c}_t^{KV} \in \mathbb{R}^{d_c}, \quad (1)$$

where $d_c \ll n_h d_h$ is the latent rank (e.g., $d_c = 512$ for DeepSeek-V2-Lite). During inference, content keys/values are reconstructed only implicitly, while positional information is carried by a decoupled RoPE key. Thus, the physical cache stores only $(\mathbf{c}_t^{KV}, \mathbf{k}_t^R)$ per token—a total of $d_c + d_R = 576$ scalars for DeepSeek-V2-Lite—compared with the $2n_h d_h = 4096$ scalars required by standard MHA, achieving roughly a $7\times$ memory reduction.

Weight Absorption at Inference. Materialising expanded keys/values for all cached tokens at every decoding step would nullify this saving. MLA avoids this through *weight absorption*: a portion of the key up-projection is absorbed into the query projection,

$$\tilde{\mathbf{q}}_{t,i} = W^{UK^\top} \mathbf{q}_{t,i}^C \in \mathbb{R}^{d_c}, \quad (2)$$

so content attention can be computed directly against cached latents:

$$\mathbf{q}_{t,i}^{C^\top} \mathbf{k}_{j,i}^C = \mathbf{q}_{t,i}^{C^\top} W^{UK} \mathbf{c}_j^{KV} = \tilde{\mathbf{q}}_{t,i}^\top \mathbf{c}_j^{KV}. \quad (3)$$

The same idea applies on the value path: aggregation is first done in latent space, then projected once. This *zero-decompression inference* property is fundamental to CalveKV: importance scoring, eviction, and attention all operate directly on $\mathbf{c}^{KV}$, without ever expanding the stored representations.

KV Cache Compression

Despite MLA’s architectural efficiency, serving LLMs in ultra-long-context regimes ($T \gg 32k$) still strains GPU memory. A complementary line of work studies *cache compression*, largely orthogonal to architectural changes.

Token Eviction. Token-eviction methods remove KV entries with low estimated future utility. Zhang et al.(2023) show that a small set of *heavy hitters* captures most cumulative attention and maintain a fixed budget of such tokens (H_2O). SnapKV (Li et al. 2024) uses a short pre-generation observation window to select representative tokens, while PyramidKV (Cai et al. 2024) allocates progressively smaller budgets in deeper layers based on layer-wise sparsification. StreamingLLM (Xiao et al. 2024) further shows that a small set of *attention sink* tokens (often leading positions) should always be retained, which aligns with CalveKV’s sink-protection design.

Their shared limitation is reliance on *expanded* key–value tensors. In MLA, this would require decompressing $\mathbf{c}_t^{KV}$ into $\mathbf{k}_t^C$ and $\mathbf{v}_t^C$ for scoring, partially restoring the memory cost MLA is meant to avoid.

Quantization. Quantization reduces per-entry storage through lower precision. KVQuant (Hooper et al. 2024) reports sub-2-bit representations with non-uniform per-channel quantization, and HqeKV (Yang et al. 2024) jointly optimizes precision and eviction under a unified importance objective. However, because quantization preserves token count, it does not reduce the $\mathcal{O}(T)$ decoding bandwidth cost.

Residual-Compensated Eviction: IndexMem

IndexMem (Anonymous 2024) mitigates eviction-induced information loss by compressing the representations of evicted tokens into a compact *latent memory module* that is injected as a residual bias into subsequent attention outputs. However, IndexMem operates on conventional expanded KV tensors: applying it within an MLA system would require first decompressing $\mathbf{c}_t^{KV}$ into full-rank $\mathbf{k}^C, \mathbf{v}^C$, partially negating MLA’s $7\times$ memory advantage and introducing additional inference-time state. CalveKV avoids this by performing all scoring and truncation natively in the compressed latent space, with no auxiliary data structures.

Summary. All existing cache compression methods were designed for uncompressed KV tensors and require decompression overhead within an MLA system. CalveKV fills this gap by operating intrinsically in the latent space, enabling zero-decompression eviction.

Method

Overview

CalveKV is a latent-space cache eviction method designed for MLA inference. For a given prefill data block, the method stores each token as a compressed latent cache entry containing RoPE components, computes importance directly within the latent space, and executes a unified global eviction decision.

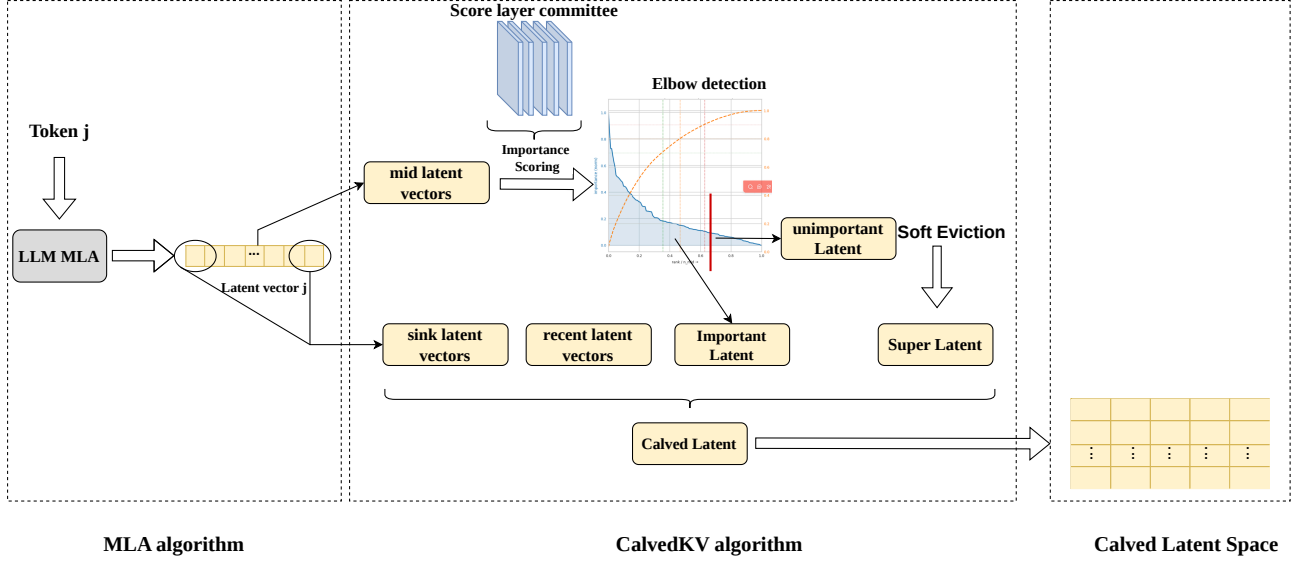


Figure 1: Overview of the CalveKV pipeline. The method performs latent-native scoring, cross-layer aggregation, and global cache eviction without reconstructing full K/V tensors.

The importance formula accounts for both the significance and the contribution of the latent vector to the output space. By default, the algorithm preserves "sink" latent vectors and recent latent vectors, while ranking intermediate latent vectors based on importance scores; experiments show that the importance distribution follows an "L-shaped" long-tail decay, prompting the use of a mechanism combining the "elbow method" and a safety net to determine which latent vectors to evict. Selected latent vectors may be merged into "super latent vectors" to preserve residual information. We observed that the importance distribution of LLM latent vectors exhibits cross-layer stability—with importance often determined at shallower layers—so we independently calculate importance scores across multiple shallow "score layers" (layers 4, 6, 8, 12, and 16), aggregate them via averaging at decision layer $D=16$, and share the resulting eviction decision across all layers. Finally, by concatenating the preserved and merged latent vectors from the beginning, end, and middle sections, we achieve significant compression of GPU memory usage.

The method contains two coupled components. Weight Absorption reuses the MLA attention path without reconstructing full K/V tensors. On top of this latent path, Query-Aware and Value-Aware scoring provides the ranking signal used by the eviction policy.

Query-Aware and Value-Aware Importance Scoring

In Multi-head Latent Attention (DeepSeek-AI 2024), the KV projection is decomposed into a compression step and an up-projection step. Given hidden state $\mathbf{h}_t$, a shared low-rank latent is computed as $\mathbf{c}_t^{KV} = W_{DKV} \mathbf{h}_t \in \mathbb{R}^R$, from which the content key and value are recovered:

$$\mathbf{k}_t^C = W_{UK} \mathbf{c}_t^{KV}, \quad \mathbf{v}_t = W_{UV} \mathbf{c}_t^{KV}. \quad (4)$$

The content (RoPE-free) attention score between positions i and j is

$$a_{ij} = \frac{(\mathbf{q}_i^C)^\top \mathbf{k}_j^C}{\sqrt{d}} = \frac{(\mathbf{q}_i^C)^\top W_{UK} \mathbf{c}_j^{KV}}{\sqrt{d}}. \quad (5)$$

Weight Absorption. By rearranging Eq. (5) via the associative law, we define the *absorbed query*:

$$\mathbf{q}_i^{\text{abs}} \triangleq W_{UK}^\top \mathbf{q}_i^C \in \mathbb{R}^{H \times R}, \quad (6)$$

so that the attention score becomes $a_{ij} = (\mathbf{q}_i^{\text{abs}})^\top \mathbf{c}_j^{KV} / \sqrt{d}$. This reformulation allows importance scoring to operate directly on the cached latent $\mathbf{c}_j^{KV} \in \mathbb{R}^{512}$ without ever materialising the expanded key $\mathbf{k}_j^C \in \mathbb{R}^{H \times 128}$, saving both memory and computation.

Causal Attention Column-Sum. Let n denote the number of candidate tokens in the prefill (excluding the protected sink and recency tokens). We construct the causally masked logit matrix for head h :

$$\hat{L}_{ij}^{(h)} = \begin{cases} \frac{(\mathbf{q}_i^{\text{abs}})^{(h)\top} \mathbf{c}_j^{KV}}{\sqrt{d}} & j \leq i, \\ -\infty & j > i, \end{cases} \quad (7)$$

and compute row-wise attention probabilities $P_{ij}^{(h)} = \text{softmax}(\hat{L}_{i,\cdot}^{(h)})_j$. The importance score of token j is its *column sum* averaged over heads:

$$s[j] = \frac{1}{H} \sum_{h=1}^H \sum_{i=j}^{n-1} P_{ij}^{(h)}, \quad (8)$$

which measures the cumulative causal attention token j receives from all subsequent positions. A high $s[j]$ indicates that many future tokens rely on j ; a low value identifies it as a candidate for eviction.

Value-Aware Modulation. High-frequency function words can attract broad but shallow attention, inflating $s[j]$ while contributing minimally to the model output. We therefore modulate the column-sum score by the ℓ_2 norm of the token’s value projection, which quantifies its magnitude of contribution in the output space:

$$\nu[j] = \|W_{UV} \mathbf{c}_j^{KV}\|_2, \quad \bar{\nu}[j] = \frac{\nu[j]}{\frac{1}{n} \sum_k \nu[k] + \varepsilon}. \quad (9)$$

The final token importance score is

$$\mathcal{I}[j] = \underbrace{s[j]}_{\text{how often attended}} \times \underbrace{\bar{\nu}[j]}_{\text{output contribution}}. \quad (10)$$

Tokens that attract sustained causal attention *and* carry large value projections are preserved; tokens with diffuse positional attention but negligible value content are discounted.

Sink and Recency Token Protection. When processing latent vectors during the prefill phase, we choose to leave a portion of the vectors at the beginning and end unprocessed. And we will scoring and evicting only those latent vectors in the middle.

The protection at the beginning is *Attention sinks* (first $N_s = 4$ tokens). Empirical analysis reveals that the leading tokens in a sequence accumulate disproportionately large cumulative attention weights across all Transformer layers. This is a phenomenon widely attributed to the “attention sink” effect (Xiao et al. 2024). The most important reason is that these tokens do not exhibit correspondingly elevated ℓ_2 norms in the latent space, their $\|\mathbf{c}^{KV}\|_2$ values are typically unremarkable, and their attention column sums under causal masking are confined to positions $i > j$, which may be sparse for early tokens. Consequently, any scoring criterion derived from latent statistics or our query-aware formulation of Eq. (10) will systematically under-value sink tokens, producing erroneous eviction. These tokens are therefore excluded from the scoring pool and unconditionally appended to the keep set.

The protection at the end is *Recency tokens* (last $N_r = 16$ tokens). Under causal attention, a token j near the end of the prefill window has been attended to by only a small number of subsequent positions, or none at all for the final token. Its causal column sum $s[j] = \frac{1}{H} \sum_h \sum_{i>j} P_{ij}^{(h)}$ is structurally small *by construction*, the attention distribution over it has not had sufficient opportunity to accumulate. Query-aware scoring therefore systematically under-estimates the importance of recent tokens, so these tokens are similarly excluded from scoring and unconditionally retained.

Formally, we can denoting the protected sets as

$$\mathcal{K}_{\text{sink}} = \{0, \dots, N_s - 1\}, \quad \mathcal{K}_{\text{recent}} = \{n - N_r, \dots, n - 1\}, \quad (11)$$

importance scoring and the subsequent adaptive budget determination are applied only to the *mid-token* interval $\mathcal{M} = \{N_s, \dots, n - N_r - 1\}$. The final keep set is the union

$$\mathcal{K} = \mathcal{K}_{\text{sink}} \cup \mathcal{K}_{\text{mid}}^* \cup \mathcal{K}_{\text{recent}}, \quad (12)$$

where $\mathcal{K}_{\text{mid}}^* \subseteq \mathcal{M}$ contains the k^* mid-tokens selected by the scoring and adaptive budget (Eq. (18)). This structure

Table 1: Gap-to-Oracle PPL analysis across single-layer eviction decisions. Baseline PPL = 6.8466; Oracle (per-layer) PPL = 8.2162. WikiText-2.

Layer	PPL	Δ vs Baseline	Gap vs Oracle
L0	26.5820	+19.7353	+18.3657
L1	13.8248	+6.9781	+5.6085
L2	15.4165	+8.5699	+7.2003
L4	9.1393	+2.2926	+0.9230
L6	8.2559	+1.4093	+0.0397
L8	8.3845	+1.5378	+0.1682
L12	7.3071	+0.4605	-0.9091
L16	7.6199	+0.7732	-0.5964

ensures that scoring noise at the sequence boundaries never contaminates the eviction decision. The budget mechanism will focus its adaptive logic exclusively on the informationally heterogeneous mid-range tokens.

Query Subsampling for Long Sequences. The full $n \times n$ attention matrix in Eq. (7) is computationally infeasible at long context: with $n = 32\text{k}$, $H = 16$ heads, and `float32` precision, it alone requires ~ 68 GB. We uniformly sample $K \ll n$ query indices from the N available query rows, and use their column-sum statistics to estimate the full attention column sum:

$$I_j \approx \frac{N}{K} \cdot \frac{1}{H} \sum_{h=1}^H \sum_{k=1}^K P_{\text{sample}(k),j}^{(h)}. \quad (13)$$

By Monte Carlo estimation, the sample expectation is proportional to the full expectation. Since eviction only requires relative ranking for Top- k selection, the global scale factor N/K does not affect ordering and can be ignored in practice. Consequently, computation and memory complexity are reduced from $\mathcal{O}(Hn^2)$ to $\mathcal{O}(HKn)$; with $K=512$, the 32k-context case requires approximately 1 GB.

Cross-Layer Shared Indices and Multi-Layer Committee Decision

A fundamental systems constraint of MLA inference mandates that all Transformer layers maintain *identical* physical KV lengths: the underlying `DynamicCache` concatenates cross-step entries layer-by-layer, and any length mismatch raises a shape error at runtime. This forces a *globally shared* eviction decision: a single index set $\mathcal{K}$ must govern all layers simultaneously. Formally, let $\mathcal{I}^{(\ell)}[j]$ denote the importance score of token j at layer ℓ . A single-layer policy selects one layer ℓ^* and defines

$$\mathcal{K} = \text{TopB}(\mathcal{I}^{(\ell^*)}) \cup \mathcal{K}_{\text{sink}} \cup \mathcal{K}_{\text{recent}}, \quad (14)$$

where $\text{TopB}(\cdot)$ retains the top- B scored mid-range tokens, and sink/recency tokens are unconditionally protected.

This constraint motivates a key design question: *which layer’s scores best represent token importance for all other layers?* Table 1 shows that single-layer eviction quality improves sharply from L0 to L6, then plateaus—suggesting that

from L4 onward, a given layer’s importance ranking is a reliable proxy for the model’s overall attention pattern. This *cross-layer persistence* property, empirically validated via Spearman rank correlation and keep-set Jaccard analysis in Section , implies that a globally shared $\mathcal{K}$ derived from shallow features already incurs only minor quality degradation.

The Oracle Paradox and Cross-Layer Consistency. A natural question is whether one should simply use a deeper, semantically richer layer (e.g., Layer 12) as the sole decision-maker, since our ablation shows L12 achieves lower PPL than either shallower layers or even the per-layer Oracle. We argue that this apparent superiority arises not from Layer 12’s feature quality alone, but from the *cross-layer consistency* enforced by shared eviction. In the Oracle baseline, each layer decides independently: token j may be retained at layer ℓ_1 yet discarded at layer ℓ_2 . This cross-layer inconsistency corrupts the residual stream—hidden states built on token j ’s contribution at ℓ_1 propagate forward, while the corresponding KV entry is absent at ℓ_2 , causing structural mismatches that accumulate across layers. Shared eviction eliminates this by enforcing a uniform token view across all layers. This is why a single shared decision (L12: PPL 7.3071) can outperform per-layer Oracle (PPL 8.2162, $\Delta = -0.9091$): Oracle’s per-layer local optimality is outweighed by its global inconsistency.

Why Not Layer 12 Alone? Despite its strong single-layer performance, relying exclusively on Layer 12 introduces two structural limitations.

(i) *Statistical instability.* Any single layer’s importance estimate is a high-variance one-sample estimate over that layer’s attention distribution. This variance cannot be reduced by choosing a “better” single layer—it reflects the irreducible stochasticity of a single-layer view.

(ii) *Representational bias.* Transformer layers encode qualitatively different levels of linguistic abstraction (Tenney, Das, and Pavlick 2019; Jawahar, Sagot, and Seddah 2019): shallow layers capture local syntax and morphology; intermediate layers integrate semantic dependencies; deep layers encode task-specific reasoning, but may over-smooth. Relying solely on Layer 12 biases the eviction decision toward tokens salient at the intermediate semantic level, systematically discounting syntactic anchors critical at shallow layers and reasoning pivots relevant at deeper layers.

Multi-Layer Committee. To jointly address both limitations, we adopt a *multi-layer committee* $\mathcal{S} = \{4, 6, 8, 12, 16\}$, where each member computes importance scores independently and the decision score is their mean:

$$\bar{\mathcal{I}}[j] = \frac{1}{|\mathcal{S}|} \sum_{\ell \in \mathcal{S}} \mathcal{I}^{(\ell)}[j]. \quad (15)$$

Averaging $|\mathcal{S}|$ estimates reduces per-token score variance by a factor of $|\mathcal{S}|$ via standard variance reduction, while the diversity of abstraction levels across committee members mitigates representational bias. The final keep set is committed at decision layer $D=16$; earlier cache entries (layers $0 \dots D-1$) are then *retroactively pruned* to the shared index set, and all subsequent layers receive the same $\mathcal{K}$. This design

introduces no extra trainable parameters and does not modify model weights.

Adaptive Eviction Budget via Knee Detection

Across virtually all code sequences, the sorted importance curve $\mathcal{I}[j]$ exhibits a pronounced *L-shaped long-tail decay*: a small token minority concentrates the dominant importance mass while the remainder carry near-zero scores. Figure 2 shows representative examples; a cross-sequence empirical analysis is provided in Section . Because the degree of concentration varies substantially across sequence types, a fixed compression ratio is inherently ill-suited—we therefore derive an *adaptive budget* from the shape of the per-sequence cumulative-mass curve $C(k)$.

Cumulative Mass Curve and Knee Point. Given the mid-token importance scores $\{\mathcal{I}[j]\}_{j=1}^n$, we shift them to non-negative values and normalise to obtain a probability mass distribution $m[j] = \mathcal{I}[j] / \sum_k \mathcal{I}[k]$. Sorting in descending order yields the concave cumulative-mass curve $C(k) = \sum_{i=1}^k m_{(i)}$, where $m_{(i)}$ is the i -th largest mass. The *knee point* of $C(k)$ is defined as the index furthest above the diagonal $(0, 0) \rightarrow (1, 1)$:

$$k_{\text{knee}} = \arg \max_k \left[C(k) - \frac{k}{n} \right]. \quad (16)$$

Geometrically, $C(k) - k/n$ measures the vertical distance from the diagonal to the curve. A concentrated importance distribution produces a steep early $C(k)$, placing the knee at a small k_{knee} and permitting aggressive eviction. A near-uniform distribution produces a shallow curve, yielding a larger k_{knee} and mandating conservative retention. The knee criterion thus adapts the budget to the content of each sequence without any learned threshold.

Coverage Floor. Knee detection alone can under-retain tokens when the distribution is approximately uniform—the knee settles near $n/2$, discarding tokens that collectively carry non-negligible importance mass. We therefore impose a *coverage floor* τ (default 0.85) that guarantees a minimum retained mass fraction:

$$k_{\text{cov}} = \min k \text{ s.t. } C(k) \geq \tau. \quad (17)$$

Final Adaptive Budget. The two estimates are combined as

$$k^* = \text{clamp} \left(\max(k_{\text{knee}}, k_{\text{cov}}), \lceil \rho_{\min} n \rceil, \lfloor \rho_{\max} n \rfloor \right), \quad (18)$$

where $\rho_{\min}$ and $\rho_{\max}$ are hard ratio bounds (defaults 0.30 and 0.98). Taking the maximum of the two estimates ensures that neither the shape-adaptive signal nor the mass-coverage guarantee is sacrificed: if the knee is overly conservative, the coverage floor tightens the budget; if the knee is overly aggressive, the coverage floor prevents excessive information loss. This formulation requires no trainable parameters and introduces negligible overhead—sorting n scores is $\mathcal{O}(n \log n)$, dominated by the $\mathcal{O}(HKn)$ importance-scoring step.

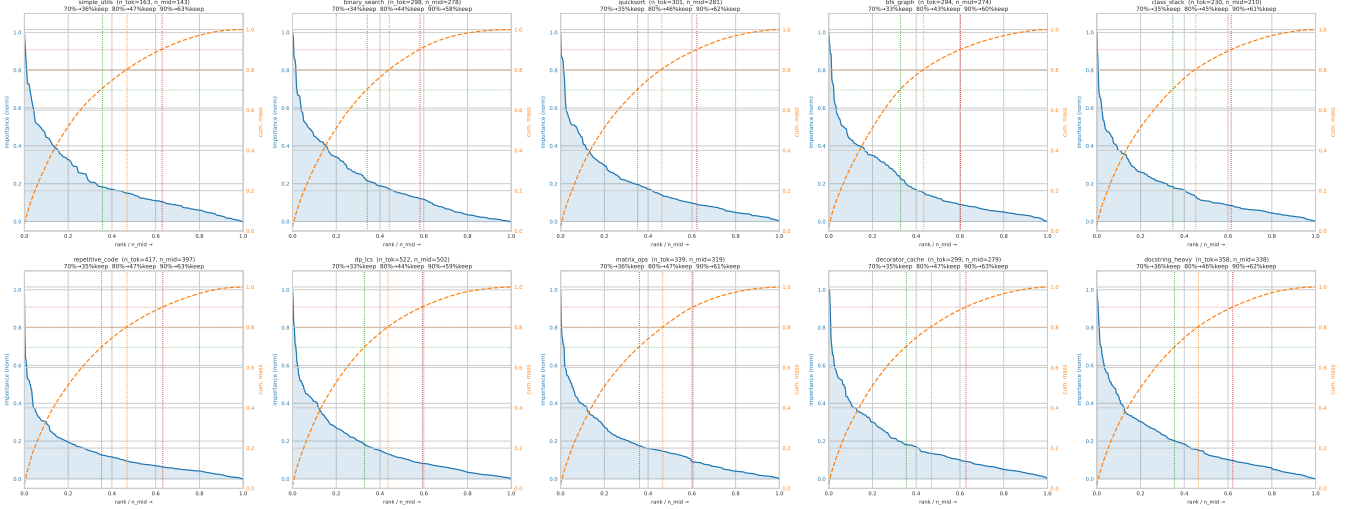


Figure 2: Importance score distributions of latent KV vectors $\mathbf{c}^{KV}$ profiled on code corpora. Each sub-figure shows sorted importance scores (blue, left axis) and the corresponding cumulative mass curve $C(k)$ (orange dashed, right axis) for a representative sequence. The distributions consistently exhibit an *L-shaped long-tail decay*: a small token minority captures the majority of importance mass, while the bulk of tokens are near-redundant. Vertical dotted lines mark the knee point k_{knee} (Eq. 16) and the coverage-floor threshold k_{cov} (Eq. 17); the adaptive budget k^* (Eq. 18) is the larger of the two.

Soft Eviction via Super Tokens

Hard eviction—permanently discarding low-ranked tokens from the cache—is irreversible: information carried by evicted tokens is lost to all subsequent decoding steps. This is particularly damaging for long-range retrieval tasks, where a token that appears informationally marginal during prefill may become highly relevant hundreds of positions later (Xiao et al. 2024).

Super-Token Construction. Rather than discarding evicted mid-tokens, CalveKV merges them into compact *super tokens* that are appended to the end of the KV cache. Consecutive evicted tokens are grouped into non-overlapping chunks of size ρ_p (pool ratio, default 4), and each chunk is reduced to a single super token via two complementary pooling strategies applied to the two components of the latent cache.

Content pooling (latent vector $\mathbf{c}$). Tokens within a chunk are combined via an importance-score-weighted softmax average:

$$\mathbf{c}_{\text{super}} = \sum_{t \in \text{chunk}} \text{softmax}\left(\frac{\mathcal{I}(t)}{T}\right) \mathbf{c}_t, \quad T = 0.1, \quad (19)$$

where T is a sharpness temperature. A low T concentrates weight on the highest-scored token within the chunk, preserving its dominant latent signal; higher T approaches a uniform average over the chunk.

Position pooling (RoPE key $\mathbf{k}^R$). Positional components are averaged uniformly:

$$\mathbf{k}_{\text{super}}^R = \frac{1}{\rho_p} \sum_{t \in \text{chunk}} \mathbf{k}_t^R. \quad (20)$$

Importance weighting is intentionally *not* applied to positional components: RoPE encodes absolute positions through phase rotations in the complex plane, and a biased phase average would distort the rotary coherence required for correct positional interpolation during decoding. Uniform averaging preserves spatial locality while producing a valid positional representation for the merged chunk.

Cache Layout and Compression Benefit. After eviction and merging, the physical KV cache for each layer is laid out as:

$$\underbrace{[\text{sink} \parallel \text{kept-mid} \parallel \text{recent}]}_{\text{preserved tokens}} \parallel \underbrace{[\mathbf{c}_1^{\text{super}}, \dots, \mathbf{c}_{n_s}^{\text{super}}]}_{\text{super tokens}}, \quad (21)$$

where $n_s = \lfloor n_{\text{evicted}} / \rho_p \rfloor$. The protected sink and recency regions remain unmodified. Super tokens are treated as ordinary cache entries during autoregressive decoding—no special attention masking or retrieval mechanism is required, and the attention computation path is entirely unchanged. The net effect is an additional ρ_p -fold reduction in the memory footprint of evicted tokens: n_{evicted} tokens are replaced by $n_{\text{evicted}} / \rho_p$ super tokens, achieving further compression beyond the primary eviction step without permanently discarding any prefill context.

Experiments

Experimental Setup

Model and Hardware. All experiments are conducted on DeepSeek-Coder-V2-Lite (DeepSeek-AI 2024), a 16-billion-parameter MLA-based code language model supporting a 128K-token context window and employing a 27-layer Transformer backbone with $d_c = 512$ KV latent rank. All inference

runs are performed on a single NVIDIA GPU with 48 GB of device memory under PyTorch 2.3.0 and CUDA 12.1. No model weights are modified; CalveKV operates purely at the inference level.

Compared Configurations. We evaluate the following configurations. *baseline*: standard MLA inference with no eviction. *committee_c85* / *committee_c90*: the full CalveKV pipeline with multi-layer committee scoring ($\mathcal{S} = \{4, 6, 8, 12, 16\}$), adaptive budget at coverage floor $\tau = 0.85$ / $\tau = 0.90$, and soft eviction via super-token pooling ($\rho_p = 4$). *hard_evict_c90*: committee scoring with $\tau = 0.90$ but *hard* eviction (no super-token aggregation), isolating the contribution of soft eviction. *single_L6* / *single_L12*: single-layer eviction using only the L6 or L12 score, with adaptive budget ($\tau = 0.90$) and soft eviction, isolating the benefit of committee voting. *fixed_ratio_70*: a non-adaptive baseline that retains a fixed 70% of mid-range tokens regardless of the importance distribution, isolating the contribution of the adaptive budget mechanism.

Evaluation Dimensions. Our evaluation covers five complementary dimensions. (1) *Cross-layer persistence* (Section): Spearman rank correlation and Jaccard keep-set overlap across ten candidate layers, directly validating the foundational assumption of committee-based shared eviction. (2) *Language modelling quality* (Section): Suffix perplexity on code-search-net/python, measuring how well the post-eviction cache supports continuation. (3) *Long-context retrieval* (Section): multi-needle NIAH recall at context lengths of 2k and 4k tokens. (4) *Code structural understanding* (Sections and): cross-reference recall across six symbol types and HumanEval syntax-pass@1. (5) *System efficiency* (Section): KV cache compression ratio, prefill latency, Time-to-First-Token (TTFT), and peak VRAM consumption across context lengths from 1k to 12k tokens.

Cross-Layer Importance Persistence Analysis

The multi-layer committee design assumes that importance scores of a latent from an early layer reliably predict which tokens matter in later layers. We test this directly across ten layers of the 27-layer backbone: $\{L0, L1, L2, L4, L6, L8, L10, L12, L16, L20\}$.

Setup. For each test sequence, all ten layers are instrumented simultaneously via forward hooks and the full $\mathcal{I}^{(\ell)}[j]$ score vector (Eq. (10)) is computed at every layer in a single forward pass. The same query subsample (Eq. (13)) is shared across all layers to remove Monte Carlo variance from the comparison. Two metrics quantify cross-layer consistency.

Spearman rank correlation $\rho(\ell_1, \ell_2)$ measures whether the importance ordering at ℓ_1 is preserved at ℓ_2 , averaged over all test sequences.

Jaccard keep-set overlap $J(\ell_1, \ell_2)$ is the Jaccard index of the top-30% keep sets selected independently at each layer:

$$J(\ell_1, \ell_2) = \frac{|\mathcal{K}^{(\ell_1)} \cap \mathcal{K}^{(\ell_2)}|}{|\mathcal{K}^{(\ell_1)} \cup \mathcal{K}^{(\ell_2)}|}. \quad (22)$$

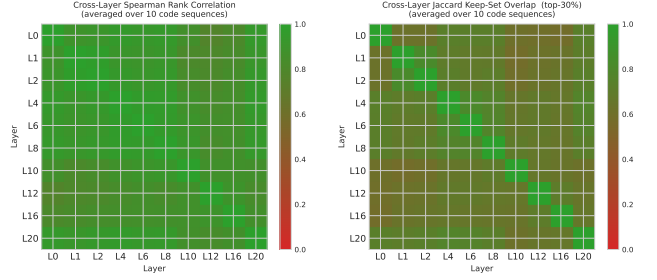


Figure 3: Cross-layer importance consistency across ten layers, averaged over ten code sequences. *Left*: Spearman rank correlation ρ . *Right*: Jaccard overlap of the top-30% keep sets. Green = high agreement; red = low agreement.

This directly measures whether two layers would reach the same eviction decision.

Figure 3 presents the 10×10 matrices.

Layers L0–L2 show substantially lower agreement with later layers than layers in the L4–L20 block, matching the elevated PPL in Table 1 when those layers are used as sole decision-makers (PPL = 26.58, 13.82, 15.42 at L0, L1, L2).

From L4 onward, all pairwise Spearman correlations exceed $\rho > 0.84$. Adjacent layers reach $\rho \approx 0.96$ (e.g. L4–L6, L6–L8). The Jaccard results follow the same pattern: L4 and L6 share $J = 0.81$ of their keep sets, falling to $J = 0.66$ at the widest gap tested (L4 vs. L16), still well above the chance baseline of $J = 0.43$.

Figure 4 tracks individual tokens across all ten layers for three sequences: *quicksort* (concentrated distribution), *binary_search* (moderate), and *repetitive_code* (near-uniform). For each sequence, the five highest-ranked, five median-ranked, and five lowest-ranked mid-tokens by L4 score are followed through all ten layers.

High-importance tokens (red) stay above the 90th percentile from L4 to L20 in all three sequences. Low-importance tokens (blue) remain near zero throughout. Mid-importance tokens (grey) fluctuate between layers; per-layer scoring noise concentrates in this ambiguous middle tier rather than at the extremes that govern the eviction boundary.

Kendall correlations between L4 and the other committee layers range from $\tau = 0.63$ at L12 to $\tau = 0.87$ at L6.

We see that L0–L2 are clearly unsuitable as scoring sources. Within the L4–L20 block, L12 shows the largest deviation from L4 ($\tau = 0.63$), confirming that no single layer generalises perfectly across all others. Averaging over $\mathcal{S} = \{4, 6, 8, 12, 16\}$ reduces per-layer noise and yields a more stable consensus ranking, consistent with *committee_c85* achieving the lowest Suffix PPL among all eviction strategies (Table 2).

Suffix PPL test

Suffix PPL evaluates how much generation quality is preserved after cache eviction. For each sample, we first run a *prefix prefill* pass, eviction may be triggered, then we compute the negative log-likelihood (NLL) on the following suffix to-

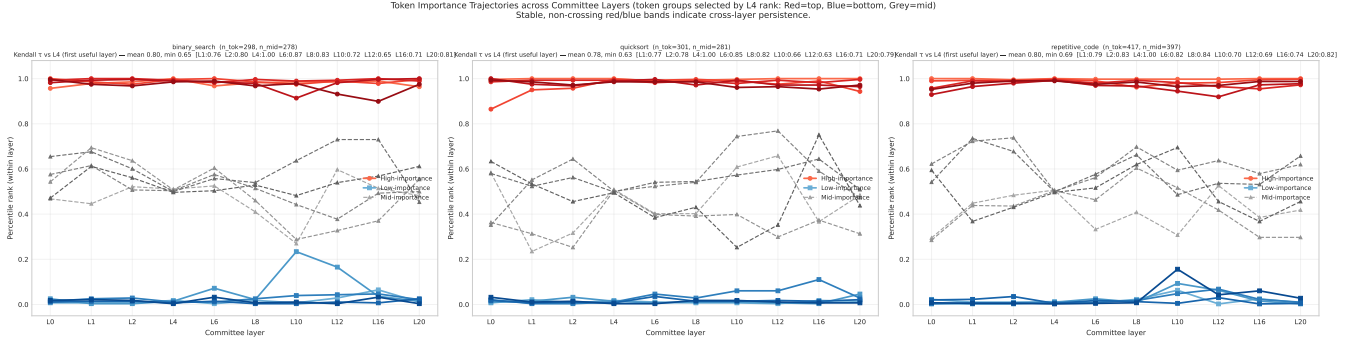


Figure 4: Per-token importance trajectories across ten layers for three code sequences. Token groups are defined by L4 percentile rank: red = top-5, blue = bottom-5, grey = middle-5. The y -axis shows within-layer percentile rank at each layer (0 = least important, 1 = most important).

Table 2: Suffix PPL on `code-search-net/python` (lower is better).

Configuration	Suffix PPL
baseline	2.546
committee_c85	3.571
committee_c90	3.611
single_L6	3.621
fixed_ratio_70	3.919
hard_evict_c90	4.066
single_L12	4.129

kens conditioned on the post-eviction KV cache. Formally, for suffix tokens $\mathbf{y}_{1:T}$:

$$\text{NLL} = -\frac{1}{T} \sum_{t=1}^T \log p(y_t \mid \mathbf{x}_{\text{prefix}}, \mathbf{y}_{<t}; \text{post-eviction cache}), \quad (23)$$

$$\text{Suffix PPL} = \exp(\text{NLL}). \quad (24)$$

Lower Suffix PPL indicates less quality degradation under the same cache budget strategy.

We use `code-search-net/python` as the evaluation corpus (12,928 tokens), with $n = 20$ samples. This metric directly tests whether the evicted cache still preserves useful predictive information for continuation. It shows whether the LLM can "speak" normally. Table 2 shows the experiments result.

Compared with baseline, all eviction variants increase Suffix PPL, this proves the expected quality-compression trade-off. Among eviction methods, `committee_c85` achieves the best PPL (3.571). `committee_c90` and `single_L6` are very close (3.611/3.621), suggesting that multi-layer committee scoring is robust and competitive in this setting. `hard_evict_c90` is clearly worse than `committee_c90` (4.066 vs 3.611), indicating that soft-eviction/super-token aggregation helps recover predictive information. `fixed_ratio_70` underperforms adaptive committee variants, supporting the value of adaptive budgeting over a rigid keep ratio. Finally, `single_L12` is the

Table 3: NIAH recall (%) at different context lengths under multi-needle protocol. Higher is better.

Context Length	baseline	committee_c90	committee_c85
2048	67%	67%	83%
4096	0%	33%	33%

weakest among tested eviction strategies (4.129), suggesting that relying on a single deeper layer is less stable than committee or L6-based decisions.

Needle-in-a-Haystack (NIAH) Evaluation

Needle-in-a-Haystack (NIAH) can evaluate long-context retrieval fidelity under cache eviction. In standard NIAH tests, both the baseline and all configurations achieve (100%) recall. So we employ a *multi-needle setup* to increase discrimination: a target integer constant is marked as "REAL ANSWER" and surrounded by 5 randomly-generated decoy constants (CACHE_SIZE, BUFFER_LENGTH, MAX_RETRIES, TIMEOUT_MS, VERSION). The model is instructed to ignore decoys and only return the value of the real constant. Strict output matching is enforced: answers without an "Answer:" prefix or embedded within variable names are rejected. This challenging variant reveals fine-grained quality differences across eviction strategies. But naive NIAH setups may miss.

The multi-needle NIAH results reveal substantial performance gaps not visible in prior saturated benchmarks. At 2k tokens, `committee_c85` achieves 83% recall, outperforming the baseline (67%) and `committee_c90` (67%). The gap widens at 4k tokens: both committee variants sustain 33% recall, while the no-eviction baseline drops to 0%, indicating systematic needle loss under eviction pressure at longer contexts.

These results demonstrate that our CalveKV’s adaptive budget and soft-eviction mechanisms provide meaningful protection for long-range retrieval, even under increased difficulty. Unlike prior NIAH evaluations that reported near-perfect recall, the multi-needle protocol with strict output matching serves as a genuinely discriminative benchmark

Table 4: Ablation: soft eviction impact on NIAH recall (%).

Configuration	ctx=2k	ctx=4k
committee_hard_c90 (no super-tokens)	100%	17%
committee_c90 (with super-tokens)	67%	33%

for eviction-induced degradation across context lengths.

Additionally, we ablate the soft-eviction component, we can see the result in table 4:

Interestingly, hard eviction performs better at 2k (100% vs 67%), suggesting that at moderate context lengths, information loss from discarding is less harmful than the noise introduced by approximate super-token pooling. However, at 4k the trend reverses: soft eviction (33%) substantially outperforms hard eviction (17%). It indicates that preserving low-rank residual information becomes critical for maintaining long-range retrieval at scale.

Code Cross-Reference Recall

Code cross-reference recall evaluates the model’s ability to retrieve variable, function, and class names defined far earlier in the context. This test complements the NIAH test. It measures long-range dependency tracking in realistic code patterns rather than synthetic needles.

Each test case embeds filler code (400+ lines of utility functions, class definitions, and constants) between the definition of a target symbol and a prompt asking the model to complete a reference to that symbol. The test verifies whether the cached KV representations preserve sufficient information for accurate recall. We conducted experiments across the following dimensions:

- *Class constant*: Reference to a constant defined within a class (`Config.DATABASE_POOL_SIZE`)
- *Module secret*: Reference to a module-level variable (`ENCRYPTION_KEY`)
- *Function recall*: Call to a user-defined function (`validate_email()`)
- *Dataclass field*: Access to a field from a dataclass (`cfg.learning_rate`)
- *Exception class*: Reference to a custom exception type (`ConnectionTimeoutError`)
- *Protocol constant*: Reference to a protocol-level constant (`MAGIC_HEADER`)

Results is shown in table 5. We observed that the eviction committee variants (`committee_c90`, `hard_evict_c90`, `fixed_ratio_70`, `single_L6`) uniformly achieve 67% recall, successfully preserving four of six symbol types. Notably, all configurations fail on *dataclass field* and *exception class* retrieval, including the baseline. It suggests that these require finer-grained type information that may be systematically under-represented in the eviction scoring.

The baseline achieves only 50% recall, failing on both function recall and exception classes, despite having access to the full KV cache. This indicates that naive full-cache

retention does not guarantee long-range dependency fidelity. The model still loses track of certain symbol definitions under long context.

Interestingly, `single_L12` (relying on a single deeper layer for importance scoring) drops to 50%, matching the baseline. This suggests that deeper layers alone provide insufficient signal for preserving syntactic anchors, such as function names or exception types. These are critical for code structure understanding. The multi-layer committee’s superior performance (67%) demonstrates that blending importance estimates from shallow, intermediate, and deep layers better preserves the diverse semantic and syntactic information needed for cross-reference resolution.

The uniformity of recall patterns across eviction strategies (committee, hard eviction, fixed ratio, single layers) at 67% suggests that the failures on dataclass fields and exception classes reflect limitations in the underlying importance-scoring formulation rather than eviction-specific degradation. This points to a direction for future work: enriching the scoring function to better capture type-definition information.

HumanEval Syntax-Pass@1

HumanEval syntax-pass@1 evaluates whether the model can generate syntactically valid Python code given a function signature and docstring. Unlike full functional correctness, this metric focuses solely on whether the generated output is parseable by the Python interpreter—a necessary precondition for any downstream execution or testing. We sample one completion per problem and report the fraction of outputs that pass `ast.parse()` without errors, using a 20-problem subset drawn from the 164-problem HumanEval benchmark (Chen et al. 2021).

The baseline achieves 70% syntax-pass@1, substantially outperforming all eviction variants, which cluster between 35–45%. This gap reflects an inherent tension between cache compression and generative coherence: KV eviction during prefill discards context that the model relies on to maintain syntactic structure across multi-line code generation.

Among eviction strategies, `fixed_ratio_70` and `single_L12` tie at 45%, narrowly ahead of `committee_c90` and `hard_evict_c90` (both 40%), while `single_L6` falls lowest at 35%. Notably, the committee-based soft eviction does not provide a clear advantage over hard eviction on this metric—both yield 40%— suggesting that super-token pooling, while beneficial for retrieval tasks (Section), does not recover syntactic generation quality lost through latent compression.

The overall degradation across all eviction variants indicates that code generation tasks, which require maintaining open syntactic structures (function bodies, indentation blocks, return statements) over hundreds of tokens, are more sensitive to context loss than retrieval tasks. This motivates future work on selectively protecting generation-critical context during the eviction phase.

System Metrics

To characterize the computational and memory efficiency of CalveKV under realistic conditions, we conduct a sys-

Table 5: Code cross-reference recall (%) by symbol type. Higher is better.

Configuration	Class Const	Module Secret	Function Recall	Dataclass Field	Exception Class	Protocol Const	Overall Score
baseline	✓	✓	—	—	—	✓	50%
committee_c90	✓	✓	✓	—	—	✓	67%
committee_c85	✓	✓	✓	—	—	✓	67%
hard_evict_c90	✓	✓	✓	—	—	✓	67%
fixed_ratio_70	✓	✓	✓	—	—	✓	67%
single_L6	✓	✓	✓	—	—	✓	67%
single_L12	✓	✓	—	—	—	✓	50%

Table 6: HumanEval syntax-pass@1 (%) on a 20-problem subset. Higher is better.

Configuration	Pass Count	Syntax-Pass@1
baseline	14/20	70%
committee_c90	8/20	40%
hard_evict_c90	8/20	40%
fixed_ratio_70	9/20	45%
single_L6	7/20	35%
single_L12	9/20	45%

tematic profiling study across a range of context lengths ($\{1024, 2048, 4096, 6144, 8192, 12288\}$ tokens). Due to the limitation of our laboratory, the maximum context is 12k in our test, but it is enough to prove the effectiveness of CalveKV. For each configuration, we measure four indicators: (i) *KV compression ratio*, defined as the fraction of latent cache entries retained after eviction relative to the full cache; (ii) *prefill latency* (ms), the wall-clock time to process the entire input prompt; (iii) *Time-to-First-Token* (TTFT, ms), the latency from prompt completion to the emission of the first generated token; and (iv) *peak VRAM consumption* (GB), recorded at the end of the prefill phase. All experiments are conducted on a single NVIDIA GPU with 48 GB of device memory under PyTorch 2.3.0 and CUDA 12.1. We have three configurations compared with baseline.

Table 7 and Figure 5 report the results. We observe the following.

The compression ratio is the most direct and important indicator in the experiment. All CalveKV variants achieve consistent compression across context lengths. `committee_c90` retains approximately 68–70% of latent cache entries, `committee_c85` retains 62–64%, and `committee_hard_c90` achieves the most aggressive reduction at 57–60%. Crucially, the compression ratio remains stable as context length scales from 1k to 12k tokens. It confirms that the adaptive budget mechanism responds consistently to the importance distribution across diverse sequence lengths rather than exhibiting length-dependent drift.

All CalveKV variants substantially reduce TTFT relative to the baseline. `committee_c90` achieves TTFT of 46–50 ms across all context lengths, compared to 57–77 ms for the baseline—a reduction of approximately 20–36%. `committee_c85` and `committee_hard_c90` exhibit comparable TTFT profiles at 47–52 ms. This improvement

arises because the first generated token requires a full attention sweep over all cached latent vectors, so a smaller retained cache directly reduces this decode step. Notably, TTFT savings are consistent across context lengths, suggesting that the eviction overhead itself is negligible relative to the attention computation saved.

Prefill latency is broadly comparable between CalveKV variants and the baseline at matched context lengths. We worried whether CalveKV will increase the Prefill latency significantly, because CalveKV performs eviction after the full prefill attention pass, the cost of importance scoring and index selection (dominated by the $\mathcal{O}(HKn)$ sampled attention computation with $K=512$ query samples). The test results show that CalveKV introduces no perceptible overhead at the scales tested. This confirms the design goal that CalveKV’s eviction pipeline adds negligible computational cost to the prefill stage.

We observe that prefill latency is broadly comparable between CalveKV variants and the baseline at context lengths of 2k tokens and above. Interestingly, at the context ($n=1024$), all CalveKV variants exhibit a substantial prefill reduction (212–260 ms versus 415 ms for the baseline, a 37–49% decrease). We attribute this to two complementary factors.

First, the baseline figure at $n=1024$ is inflated by *CUDA cold-start overhead*. On the first kernel execution in each experimental run, PyTorch incurs one-time costs for kernel compilation, CUDA context initialization, and GPU memory-pool allocation, all of which are amortized across subsequent runs. Because the baseline is always profiled first in our test, it absorbs this initialization penalty disproportionately at short contexts, where the actual compute time is otherwise negligible.

Second, CalveKV’s query subsampling computes a sampled attention matrix of shape $K \times n$ (with $K=512$) rather than the full $n \times n$ matrix. At $n=1024$, the sampled pass requires roughly half the floating-point operations of the standard prefill attention, and fits more favorably in L2 cache, yielding a lower effective memory-bandwidth pressure. At longer contexts (4k–12k), the baseline prefill time is dominated by the $\mathcal{O}(n^2)$ attention sweep, making the $\mathcal{O}(HKn)$ scoring overhead negligible in comparison; the two curves therefore converge. This also confirms that CalveKV introduces no significant prefill overhead under normal operating conditions.

Peak VRAM consumption is nearly identical across all configurations at each context length (differences $\leq$

Table 7: System metrics across context lengths. Compress = retained KV fraction (%); Prefill = prefill latency (ms); TTFT = time-to-first-token (ms); VRAM = peak GPU memory (GB). Lower is better for Prefill, TTFT, and VRAM.

Configuration	Context (tokens)	Compress (%)	Prefill (ms)	TTFT (ms)	VRAM (GB)
baseline	1024	100.0	415	77.0	30.64
baseline	2048	100.0	276	57.0	30.63
baseline	4096	100.0	486	57.2	33.19
baseline	6144	100.0	815	62.0	37.75
baseline	8192	100.0	1185	56.8	44.08
baseline	12288	100.0	2421	56.7	62.05
committee_c90	1024	69.8	260	49.8	30.08
committee_c90	2048	69.1	272	48.4	30.69
committee_c90	4096	68.4	473	46.6	33.25
committee_c90	6144	68.6	777	46.4	37.76
committee_c90	8192	68.2	1197	46.5	44.05
committee_c90	12288	67.4	2423	46.5	61.95
committee_c85	1024	64.2	219	46.9	30.07
committee_c85	2048	62.5	298	46.6	30.68
committee_c85	4096	62.4	483	47.1	33.22
committee_c85	6144	61.9	782	52.0	37.75
committee_c85	8192	62.2	1204	50.9	44.04
committee_c85	12288	61.9	2442	47.4	61.92
committee_hard_c90	1024	59.5	212	46.9	30.07
committee_hard_c90	2048	58.4	275	56.5	30.68
committee_hard_c90	4096	58.0	477	47.2	33.22
committee_hard_c90	6144	57.7	779	51.9	37.74
committee_hard_c90	8192	57.4	1187	47.0	44.03
committee_hard_c90	12288	56.9	2433	47.4	61.90

0.15 GB). This is an expected result. Peak memory during prefill is dominated by model parameters and intermediate activations rather than the KV cache itself, whose size at the end of prefill constitutes a minor fraction of total device memory at these context lengths. The true memory savings of CalveKV are realized during the decoding stage, where a compressed KV cache directly reduces the $\mathcal{O}(TLd_c)$ memory footprint proportionally to the retained fraction. This regime can not be captured by prefill-phase peak measurements but can be directly reflected by the compression ratios mentioned above.

Importance Score Distribution Analysis

To empirically validate the adaptive budget design of CalveKV, we profiled per-sequence importance scores $\mathcal{I}[j]$ across ten representative code sequences spanning diverse abstraction levels—from simple utility functions and sorting algorithms to graph traversal, dynamic programming, and docstring-heavy stubs. Figure 2 presents all ten sorted-score curves and Figure 6 presents their corresponding cumulative-mass curves $C(k)$ overlaid on a normalised token-rank axis.

We observe that across all ten sequences, the sorted score curve falls steeply from a sharp peak and flattens toward zero well before the full token budget is exhausted. In every case, fewer than 30% of mid-range tokens account for more than 80% of the total importance mass. This confirms that the L-shaped long-tail decay described in Section 3 is a robust structural property of MLA importance scores on code

corpora. And it is not an artefact of any particular program structure.

While all curves share the same qualitative shape, the degree of concentration varies substantially. Algorithmically dense sequences (`quicksort`, `dp_lcs`, `decorator_cache`) exhibit steeper decays and require only 15–25% of tokens to reach 80% cumulative mass. By contrast, repetitive or comment-heavy sequences (`repetitive_code`, `docstring_heavy`) produce shallower curves, requiring 35–50% retention for the same coverage level. So a fixed compression ratio cannot simultaneously respect both extremes. A ratio calibrated for concentrated sequences over-evicts the flat-distribution cases, while a ratio calibrated for uniform sequences wastes memory on the concentrated ones. The variability in Figure 6 therefore directly motivates the knee-detection and coverage-floor mechanism of Eq. (18).

For the flattest curves, the knee point k_{knee} (Eq. (16)) alone would settle near $k/n \approx 0.45$, discarding a non-negligible fraction of uniformly-distributed importance mass. The coverage floor k_{cov} (Eq. (17), $\tau = 0.85$) acts as a safety net, raising the effective budget to at most the 85% mass threshold for these sequences. Conversely, for the steepest curves the coverage floor is already satisfied at $k/n < 0.20$, so the knee criterion takes over and permits aggressive eviction without triggering the floor. This complementary interaction validates the $\max(k_{\text{knee}}, k_{\text{cov}})$ formulation of the adaptive budget. In the following experiment, τ is a param-

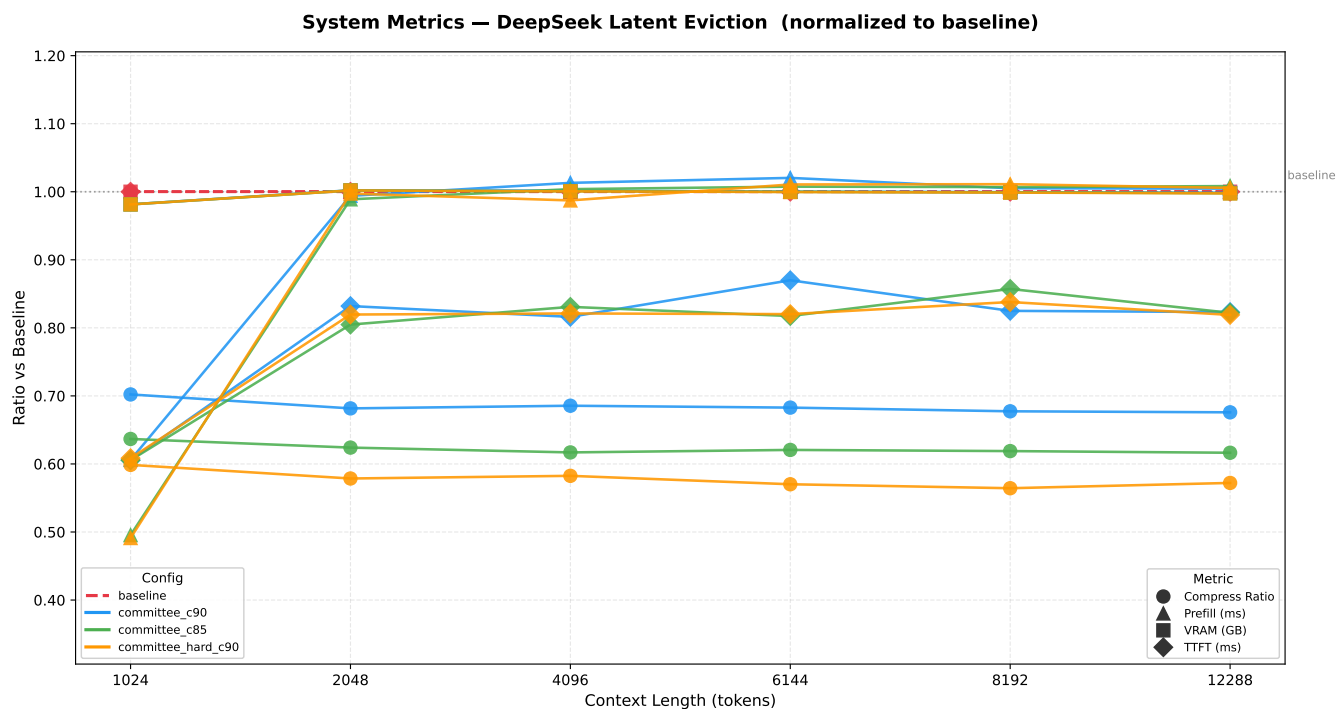


Figure 5: System metrics of CalveKV variants versus the MLA baseline across context lengths. Top: KV cache compression ratio (%). Middle: Time-to-First-Token (TTFT, ms). Bottom: peak VRAM consumption (GB).

eter which can be chosen according to different models or tasks. For example, `committee_c85` means τ is 0.85 and `committee_c90` means τ is 0.9.

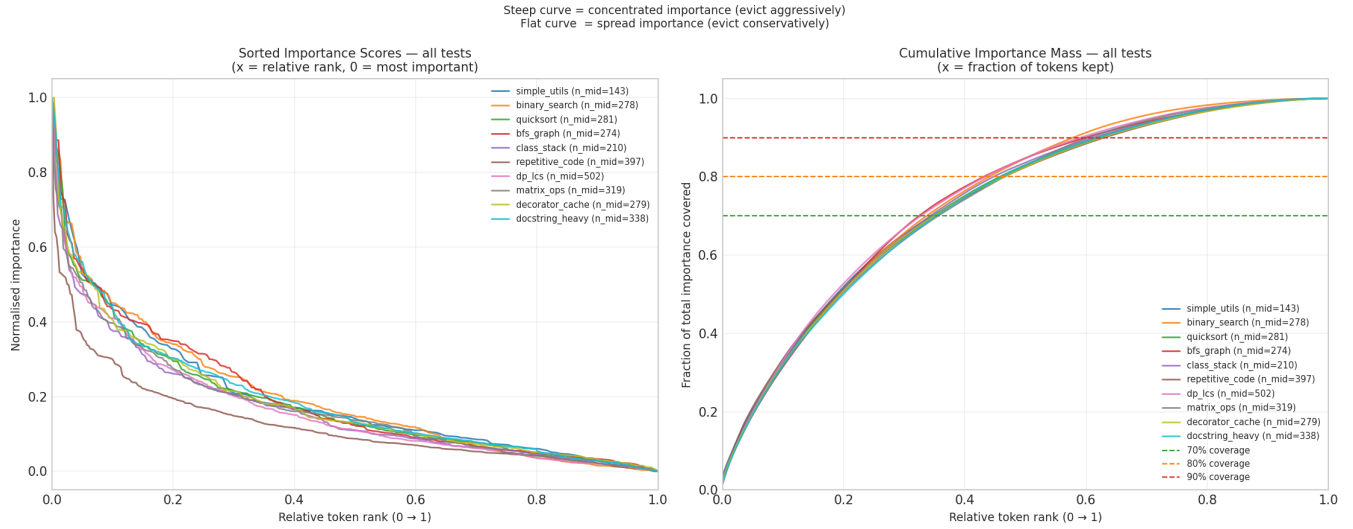


Figure 6: Overlay of normalised importance-score curves (left) and cumulative-mass curves $C(k)$ (right) across ten code sequences of varying abstraction level. The x -axis is the relative token rank after sorting in descending importance order. Dashed horizontal lines mark the 70%, 80%, and 90% coverage thresholds. Steep score curves indicate concentrated importance and permit aggressive eviction; flat curves indicate near-uniform distributions and mandate conservative retention.

References

- Anonymous. 2024. IndexMem: Residual-Compensated Memory for Evicted Token Information in Long-Context LLM Inference. Technical report.
- Cai, Z.; et al. 2024. PyramidKV: Dynamic KV Cache Compression Based on Pyramidal Information Funneling. In *arXiv preprint*.
- Chen, M.; Tworek, J.; Jun, H.; Yuan, Q.; Pinto, H. P. d. O.; Kaplan, J.; Edwards, H.; Burda, Y.; Joseph, N.; Brockman, G.; Ray, A.; Puri, R.; Krueger, G.; Petrov, M.; Khlaaf, H.; Sastry, G.; Mishkin, P.; Chan, B.; Gray, S.; Ryder, N.; Pavlov, M.; Power, A.; Kaiser, L.; Bavarian, M.; Winter, C.; Tillet, P.; Such, F.; Cummings, D.; Plappert, M.; Chantzis, F.; Barnes, E.; Herbert-Voss, A.; Guss, W.; Nichol, A.; Paino, A.; Tezak, N.; Tang, J.; Babuschkin, I.; Balaji, S.; Jain, S.; Saunders, W.; Hesse, C.; Carr, A.; Leike, J.; Achiam, J.; Misra, V.; Morikawa, E.; Radford, A.; Knight, M.; Brundage, M.; Murati, M.; Mayer, K.; Welinder, P.; McGrew, B.; Amodei, D.; McCandlish, S.; Sutskever, I.; and Zaremba, W. 2021. Evaluating Large Language Models Trained on Code. *arXiv preprint arXiv:2107.03374*.
- DeepSeek-AI. 2024. DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model. Technical report.
- DeepSeek-AI. 2025. DeepSeek-V3 Technical Report. Technical report.
- Hooper, C.; et al. 2024. KVQuant: Towards 10 Million Context Length LLM Inference with KV Cache Quantization. *ArXiv preprint*.
- Jawahar, G.; Sagot, B.; and Seddah, D. 2019. What Does BERT Learn about the Structure of Language? In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, 3651–3657. Florence, Italy: Association for Computational Linguistics.
- Li, Y.; et al. 2024. SnapKV: LLM Knows What You are Looking for before Generation. *ArXiv preprint*.
- Tenney, I.; Das, D.; and Pavlick, E. 2019. BERT Rediscovered the Classical NLP Pipeline. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, 4593–4601. Florence, Italy: Association for Computational Linguistics.
- Xiao, G.; et al. 2024. Efficient Streaming Language Models with Attention Sinks. In *ICLR*.
- Yang, Y.; et al. 2024. HqKV: Hybrid Quantization and Eviction for Efficient KV Cache Compression. *ArXiv preprint*.
- Zhang, Z.; et al. 2023. H₂O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models. In *NeurIPS*.